

Driver Guard Project Architecture Overview

This document serves as a complete breakdown of what we have built so far, explaining every technology, language, and structural component that makes up your application.

1. Programming Languages Used

- **JavaScript (ES6+)**: The primary logic language driving the entire platform. Both the frontend web interactions and backend servers are written in JS.
- **JSX (JavaScript XML)**: A syntax extension to write HTML-like elements inside JavaScript, used across all React components.
- **HTML5**: Used minimally (mainly just `index.html`) to serve as the structural anchor for React to render into.
- **CSS3 (Vanilla CSS)**: Used to style the app without relying on heavy frontend frameworks like Tailwind. We utilized complex custom variables, flexbox, grid, glassmorphism designs, and micro-animations (e.g., in `animations.css` and `index.css`).

2. System Architecture Components

A. The Frontend (Client-Side Interface)

This is what the driver interacts with on their phone.

- **React.js (v19)**: The core user interface framework powering the screens, states, and buttons. It creates a snappy Single Page Application.
- **React-Router-DOM**: Manages page navigation (Dashboard, Profile, Settings, Login, Register, Forgot Password) without refreshing the browser.
- **React-Leaflet & Leaflet**: A mapping library used to visualize real-time GPS locations logically on the screen.
- **Lucide-React**: The icon pack providing all the sleek, modern SVG icons seen in the app.
- **Vite**: The exceedingly fast build tool that compiles and runs our React code during development.
- **Capacitor JS**: The "Bridge". This is what takes our HTML/CSS/JS web application and wraps it into a native Android application container, allowing it to be compiled into an `.apk` file that we install on an Android phone.

B. The Backend (Server-Side Logic)

This is the hidden engine running in the background listening for hardware triggers.

- **Node.js**: The JavaScript runtime executing the server files (`server.js` , `start_both.js`).
- **Express.js**: A lightweight web framework used in `server.js` to create an API. Specifically, we built an endpoint: `POST /api/trigger-emergency` .
- **Localtunnel**: Because your Raspberry Pi is on a different network and needs to talk to your computer, Localtunnel generates a temporary public `https://` web address that points securely to your local Express server. We automated this using `start_both.js` .

C. The Cloud Database & Authentication

- **Firebase Authentication**: Handles secure driver sign-ups, log-ins, and password resets securely.
- **Firebase Realtime Database**: Stores your data instantly in the cloud. We use this to save Profile Information (License number, vehicle number, address), App Settings, and the Emergency Contact (name and phone number) mapped directly to that driver's unique ID.

D. The Middleware & APIs

Middleware acts as a plugin connecting different parts together seamlessly.

- **CORS (Cross-Origin Resource Sharing):** Security middleware used in Express so it accepts external HTTP requests from the Raspberry Pi or React App without browser security errors blocking it.
- **Twilio API:** The messaging powerhouse. When the Express API server receives a trigger, it uses the Twilio Node.js SDK to securely generate and send a WhatsApp distress message to the saved emergency contact phone number stored dynamically.

3. How It All Connects (The Flow)

1. **Sensors -> Server:** A Raspberry Pi hardware system (monitor/camera/accelerometer) detects that the driver is drowsy or involved in an impact.
2. **Raspberry Pi -> Internet:** The Raspberry Pi sends an HTTP `POST` alert to your public `localhost` URL.
3. **Internet -> Express API:** Localtunnel forwards that request internally into your running Express `<server.js>`.
4. **API -> Twilio:** Express processes the alert and commands Twilio to immediately dispatch an automated WhatsApp template message to the driver's relatives.
5. **App Sync:** Meanwhile, the React frontend stays locally connected to Firebase, letting the driver comfortably manage settings and ensuring the correct contact is always alerted.

4. Software Programs Used locally

- **Visual Studio Code (VS Code):** Our code editor.
- **Node Package Manager (npm / npx):** Used to download open-source libraries (like React, Capacitor, Express) and run scripts.
- **Android Studio / Gradle:** Utilized as the heavy-duty compiler to turn the Capacitor assets into a runnable `.apk` binary container.